

Task 2: The Distribution of the Sample Mean

COURSE NAME

CSDS 413 Introduction to Data Analysis

Authors:

Jacob Anderson

October 13, 2025

Contents

Context	2
1 Part A: Data Generation	3
2 Part B: Sample Means and Visualization	7
3 Part C: Normal Approximation via CLT	12
4 Part D: Distribution Comparison with KS-Test	17

Context

In this task, you will we gain insight about the Central Limit Theorem. Consider three random variables:

- (a) A **Uniform distribution** on $[0.25, 1.25]$
- (b) A **Power-Law distribution** (with parameter $\alpha = 3$)

1 Part A: Data Generation

- For each distribution, generate **replicates of data** with sizes 10, 100, and 1000.
- Each replicate should consist of 100 samples.
- Example: For “10 replicates,” generate 10 separate datasets, each with 100 samples.
- For one replicate from each distribution, plot the distribution of the sample to ensure that it follows a uniform/power-law distribution.

Implementation notes:

- In **Python**, use `numpy.random.uniform` and `numpy.random.power`.
- In **R**, use `runif()`, `rplcon()` (for Power, with $\text{shape1} = \alpha$, $\text{shape2} = 2.5$).

—

For each distribution, the replicates were generated according to the recommended NumPy functions, and in the case of the second distribution, I took the guidance of the Canvas announcement and used the Pareto distribution from `numpy.random.pareto` with α parameter set to 2.3. My implementation of the generation functions just wrap the corresponding NumPy calls that let me set the shape according to the replicate sizes. So for the replicate with size 10, I have a CSV with 10 columns, one for each set of 100 samples. These are the relevant utility functions:

```
def generate_uniform_replicates(low: float, high: float, sample_size: int,
                               num_replicates: int) -> np.ndarray:
    """
    Generates the replicates of uniformly distributed data.

    :param low: Lower bound of uniform distribution
    :type low: float
    :param high: Upper bound of uniform distribution
    :type high: float
    :param sample_size: Number of samples per replicate
    :type sample_size: int
    :param num_replicates: Number of replicates to generate
    :type num_replicates: int
    :returns: Array containing the replicates.
    :rtype: np.ndarray
    """
    replicates = np.random.uniform(low=low, high=high,
                                   size=(num_replicates, sample_size))
    return replicates

def generate_powerlaw_replicates(sample_size: int,
                                 num_replicates: int) -> np.ndarray:
    """
    Generates the replicates of power-law distributed data using Pareto distribution.

    :param alpha: Shape parameter for the power-law distribution
    :type alpha: float
    :param sample_size: Number of samples per replicate
    :type sample_size: int
    :param num_replicates: Number of replicates to generate
    :type num_replicates: int
    :returns: Array containing the replicates.
    :rtype: np.ndarray
    """
    replicates = np.random.pareto(a=2.3, size=(num_replicates, sample_size))
    return replicates
```

Figure 1: Data generation functions for Uniform and Power-Law distributions.

To verify that the generated replicates follow their expected distributions, I plotted the distribution of a single replicate for both the distributions. The visualization function and corresponding plots are shown below:

```
def plot_single_replicate(data: np.ndarray, dist_name: str,
                        output_filename: str):
    """
    Plots the distribution of a single replicate.

    :param data: Single replicate samples
    :type data: np.ndarray
    :param dist_name: Name of the distribution
    :type dist_name: str
    :param output_filename: Filename for the figure
    :type output_filename: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")
    plt.figure(figsize=(10, 6))

    ax = sns.histplot(data, bins=50, stat='density',
                      edgecolor='white', alpha=0.7)

    ax.set_xlabel('Value', fontsize=14, labelpad=15)
    ax.set_ylabel('Density', fontsize=14, labelpad=15)
    ax.set_title(f'Distribution of {dist_name} Replicate',
                 fontsize=16, pad=20)

    plt.tight_layout()
    plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)
```

Figure 2: Visualization function for verifying the distribution shape.

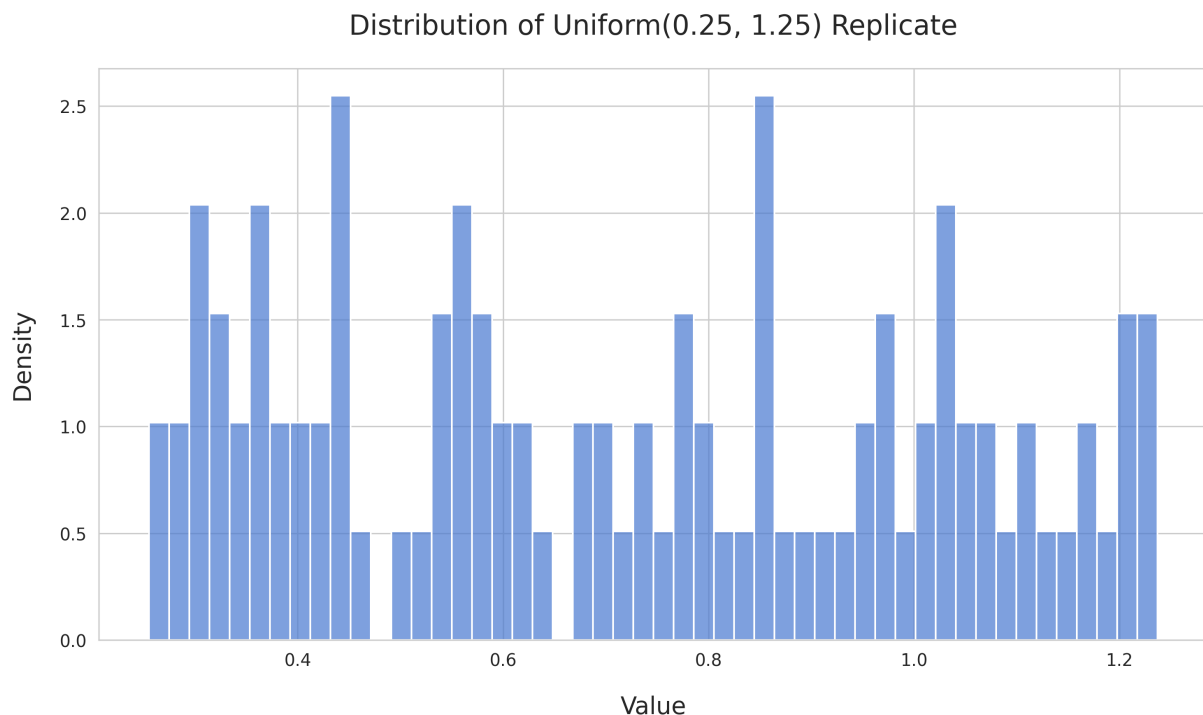


Figure 3: Distribution of a replicate from the Uniform(0.25, 1.25) distribution.

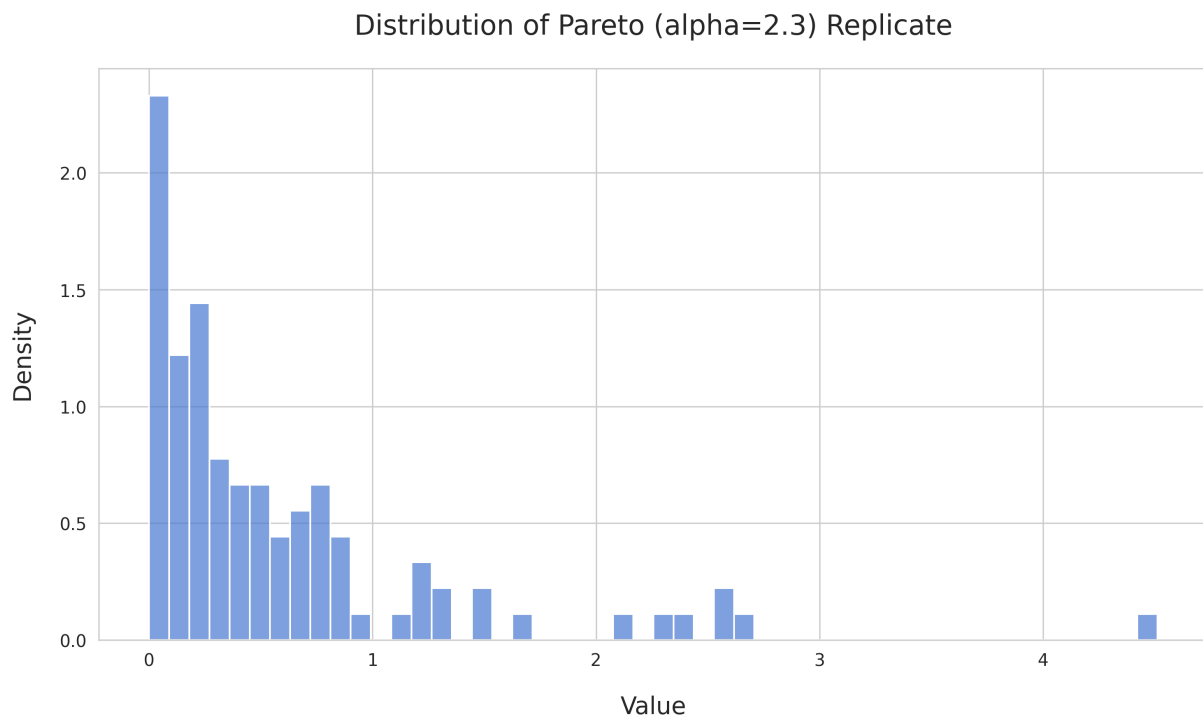


Figure 4: Distribution of a replicate from the Pareto($\alpha = 2.3$) distribution.

The Uniform replicate plot in Figure 3 shows the generally flat distribution of mass that we would expect across the interval, at least as flat as we could reasonably expect from only 100 samples.

The Pareto replicate plot in Figure 4 exhibits the characteristic right-skewed shape of a heavy-tailed power-law distribution, like the ones we have discussed in lecture. Most of the distribution mass is concentrated near zero, and a long tail extends out to larger values, showing that generally the probability density decreases with larger values, but extreme values are still present and predictable in the sample.

Regarding `np.random.power` and `np.random.pareto`, the Canvas announcement made the clarification that we are meant to explore the discrepancy in the distribution shapes these two approaches produce. According to the NumPy documentation, `np.random.power` draws samples from a power function distribution with positive exponent $a - 1$. The probability density function is:

$$P(x; a) = a \cdot x^{a-1}, \quad 0 \leq x \leq 1, \quad a > 0$$

Also I feel I should mention that the documentation literally states, "The power function distribution is just the inverse of the Pareto distribution". So given that guidance and looking at our case in the assignment, alpha is meant to be set to 3 if using `numpy.random.power`, making the PDF:

$$P(x; a) = 3x^2, \quad 0 \leq x \leq 1$$

There are definitely a lot of characteristics that distinguish this equation from the power-laws we have looked at in the lecture as well as the one defined by `numpy.random.pareto` (bounded interval, increasing density instead of decreasing, etc.), but let's look first at the Pareto distribution PDF. `np.random.pareto(a)` samples from a Pareto Type II distribution with shape parameter a and implicit scale $m = 1$, which has the probability density function:

$$p(x) = \frac{a}{(1+x)^{a+1}}, \quad x \geq 0$$

Applying our setting of $a = 2.3$ gives the PDF $p(x) = \frac{2.3}{(1+x)^{3.3}}$. So the `np.random.power` approach is described by $3x^2$ on $[0, 1]$, where the exponent is positive and the function value increases as x approaches 1, concentrating almost all probability mass near the upper bound with no capacity for extreme outliers beyond this range. In contrast, `np.random.pareto` is described by $\frac{2.3}{(1+x)^{3.3}}$ on $[0, \infty)$, where the function value decreases as x increases (since $(1+x)^{3.3}$ in the denominator grows), placing most mass near zero but maintaining enough probability in the tail to produce occasional extreme values.

This distribution produced by `numpy.random.power` seems distinctly different from what we have discussed in lecture. After having applied the settings given in the assignment instructions, the inverse relationship and subsequent effect on the distribution shape was readily obvious, but that still did not explain why `numpy.random.power` is set up in this manner; it seems inconsistent with what I have come to understand Power-Law to be from class. My understanding from the documentation and community support online has led me to believe that `numpy.random.power` generates from a power function distribution, with an exponential relationship to its alpha parameter, which is inconsistent with the general power-law relationship that we established in lecture; that is, $p(x) \propto x^{-a}$. The Pareto Type II distribution from `np.random.pareto`, on the other hand, exhibits the characteristic heavy-tail behavior where the denominator $(1 + x)^{3.3}$ creates the decreasing density function consistent with power-law relationship. They are related but distinctly different in their construction and produce starkly different distribution shapes as a result.

2 Part B: Sample Means and Visualization

- Compute the sample means for each set of replicates. You should have 9 of them, 3 for each distribution with varying sample sizes.
- For each distribution and replicate size, we want to view and compare their distributions. Which plot do you think is appropriate? Plot the appropriate plot for each replicate, please plot them separately.
- Do not confuse the distribution of sample mean with the distribution of the sample (which you plotted in Part A).

For each set of replicates, I computed the sample mean across the 100 observations within each replicate. This resulted in 6 sets of sample means: 3 for the Uniform distribution (with 10, 100, and 1000 replicates) and 3 for the Pareto distribution (with 10, 100, and 1000 replicates). I'm guessing that having 9 of them is outdated, because 3 each from the 2 distributions is 6 sample means. The implementation is shown below:

```
def compute_sample_means(replicates_csv: str) -> np.ndarray:
    """
    Computes the sample mean for each replicate.

    :param replicates_csv: Path to CSV containing replicates
    :type replicates_csv: str
    :returns: Array of sample means, one per replicate
    :rtype: np.ndarray
    """
    df = pd.read_csv(replicates_csv)
    sample_means = df.mean(axis=0).to_numpy()
    return sample_means
```

Figure 5: Function for computing sample means from replicates.

To visualize the distribution of sample means, I chose to use histograms. This choice is best because histograms effectively visualize the shape and spread of the distribution mass of sample means, allowing us to best assess whether they appear to be approaching a normal distribution as the Central Limit Theorem predicts for larger numbers of replicate sample means. Since the whole point of this task is to gain insight about CLT, this plot ought to be the most appropriate to appreciate the convergence to Normal. The plotting function is shown below:

```
def plot_sample_means_distribution(sample_means: np.ndarray, dist_name: str,
                                  replicate_size: int, output_filename: str):
    """
    Plots the distribution of sample means.

    :param sample_means: Array of sample means
    :type sample_means: np.ndarray
    :param dist_name: Name of the distribution
    :type dist_name: str
    :param replicate_size: Replicate size
    :type replicate_size: int
    :param output_filename: Output filename for figure
    :type output_filename: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")
    plt.figure(figsize=(10, 6))

    ax = sns.histplot(sample_means, bins=50, stat='density',
                      edgecolor='white', alpha=0.7)

    ax.set_xlabel('Sample Mean', fontsize=14, labelpad=15)
```



```

ax.set_ylabel('Density', fontsize=14, labelpad=15)
ax.set_title(f'Distribution of {dist_name} Sample Means of Replicate with size {replicate_size}',
            fontsize=16, pad=20)

mean_of_means = np.mean(sample_means)
ax.axvline(mean_of_means, color='red', linestyle='--', linewidth=2,
            label=f'Mean of means: {mean_of_means:.3f}')
ax.legend(fontsize=11)

plt.tight_layout()
os.makedirs("../figures", exist_ok=True)
plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)

```

Figure 6: Visualization function for plotting distribution of sample means.

These are the separate corresponding plots for each distribution and size:

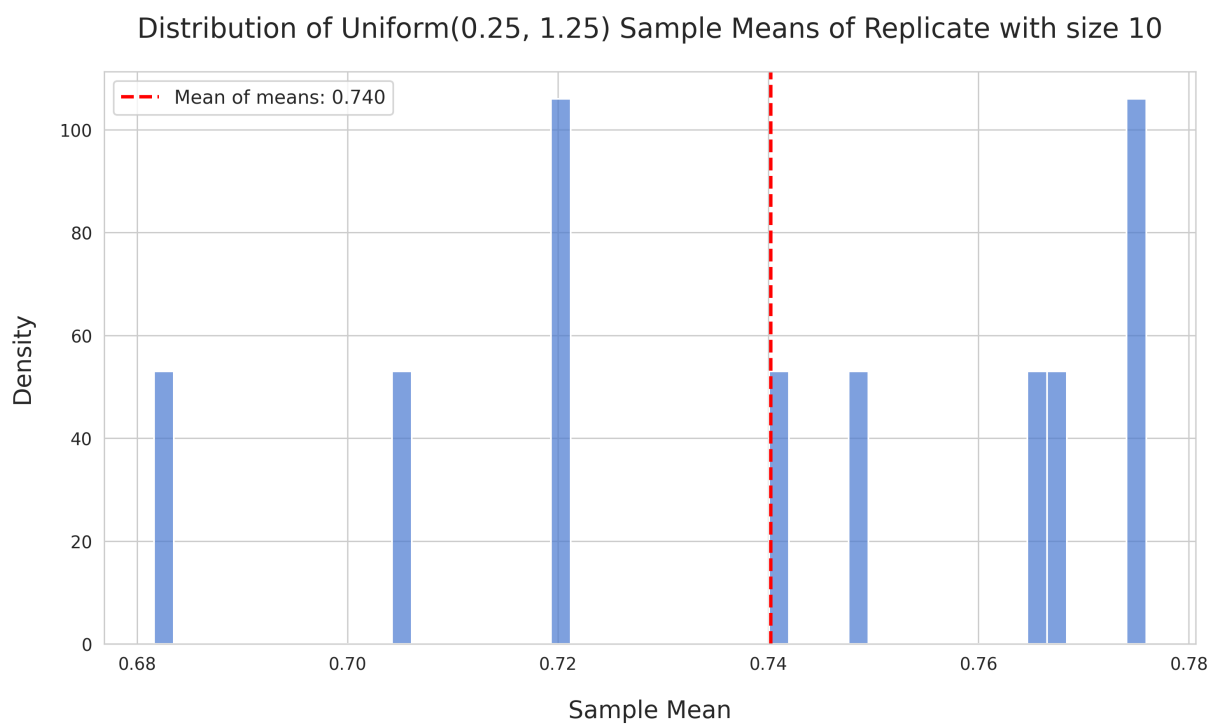


Figure 7: Distribution of sample means from Uniform(0.25, 1.25) with replicate size 10.

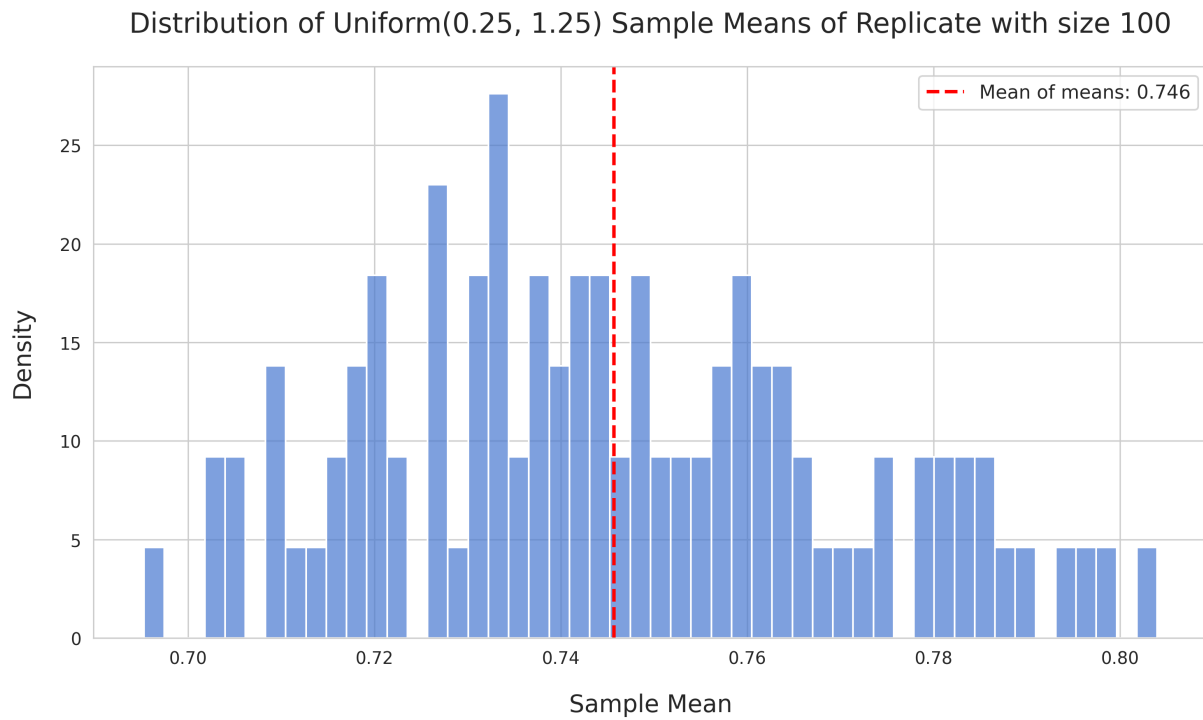


Figure 8: Distribution of sample means from Uniform(0.25, 1.25) with replicate size 100.

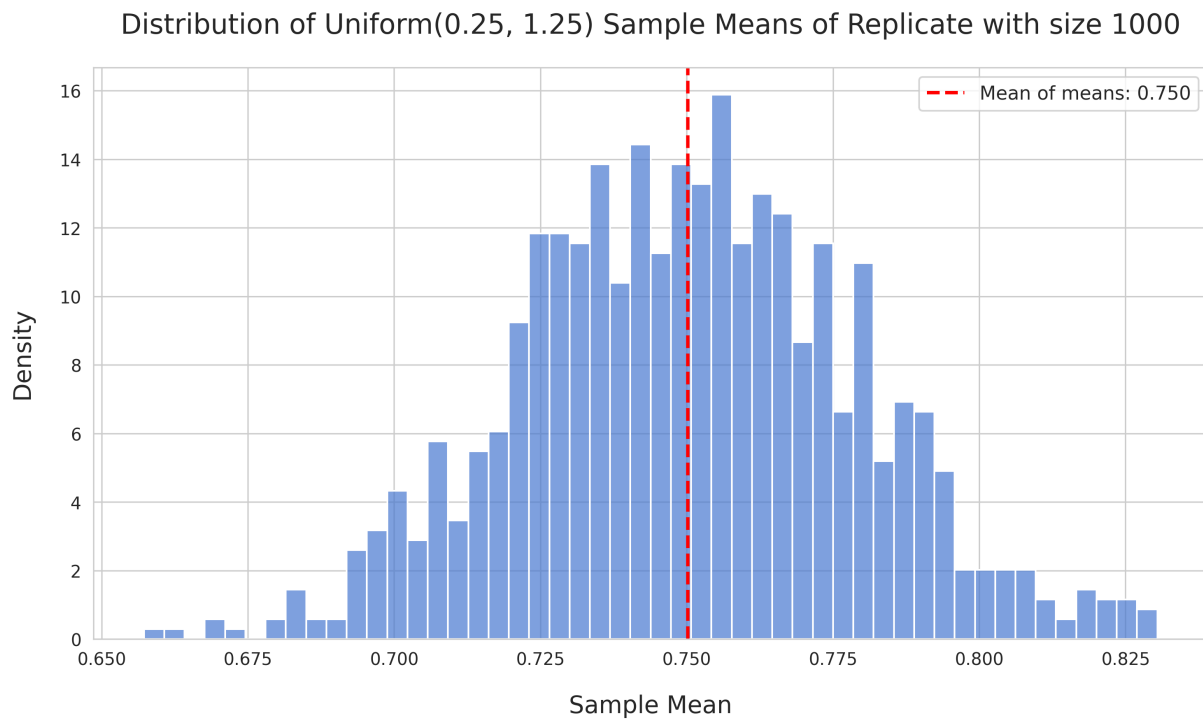


Figure 9: Distribution of sample means from Uniform(0.25, 1.25) with replicate size 1000.

Regarding the Uniform sample mean plots, we see the first plot begin with a very rough estimate of population mean from just 10 separate datasets, and we do not yet see a clear distribution. The second plot starts to take on form and shows a more accurate estimation of population mean. We can see the general shape of a Gaussian, which is then confirmed in the third plot, centered around 0.75, equal to $\frac{1.25+0.25}{2}$.

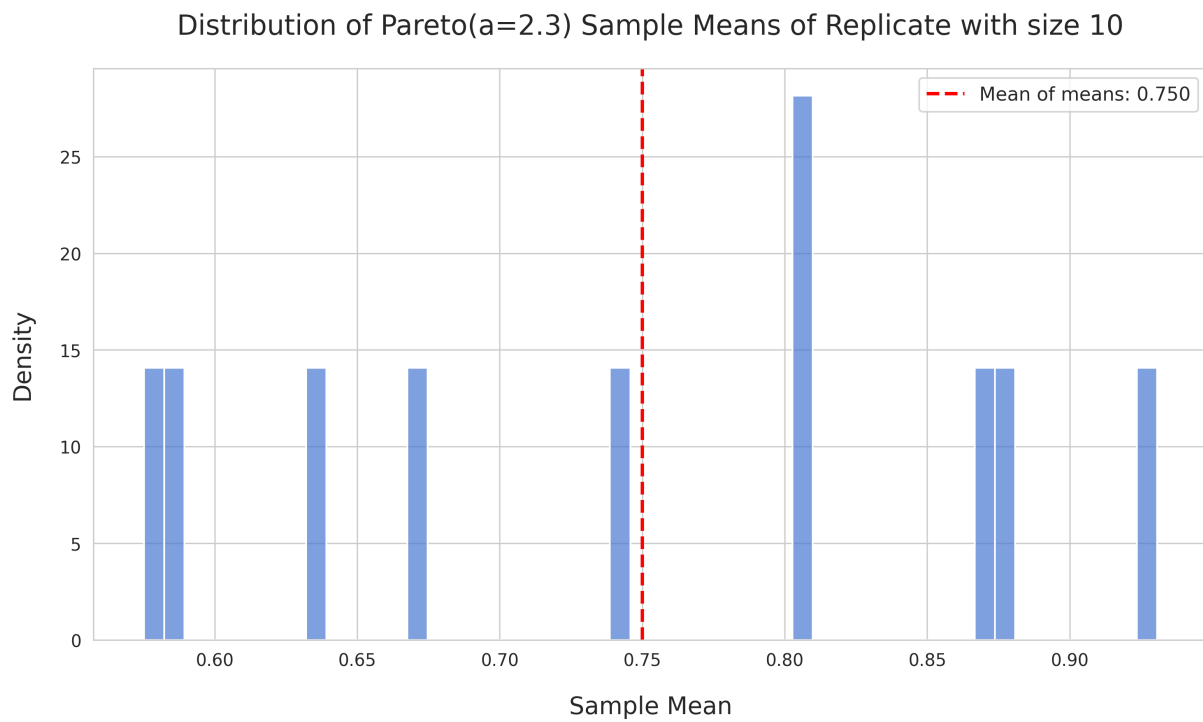


Figure 10: Distribution of sample means from Pareto($\alpha=2.3$) with replicate size 10.

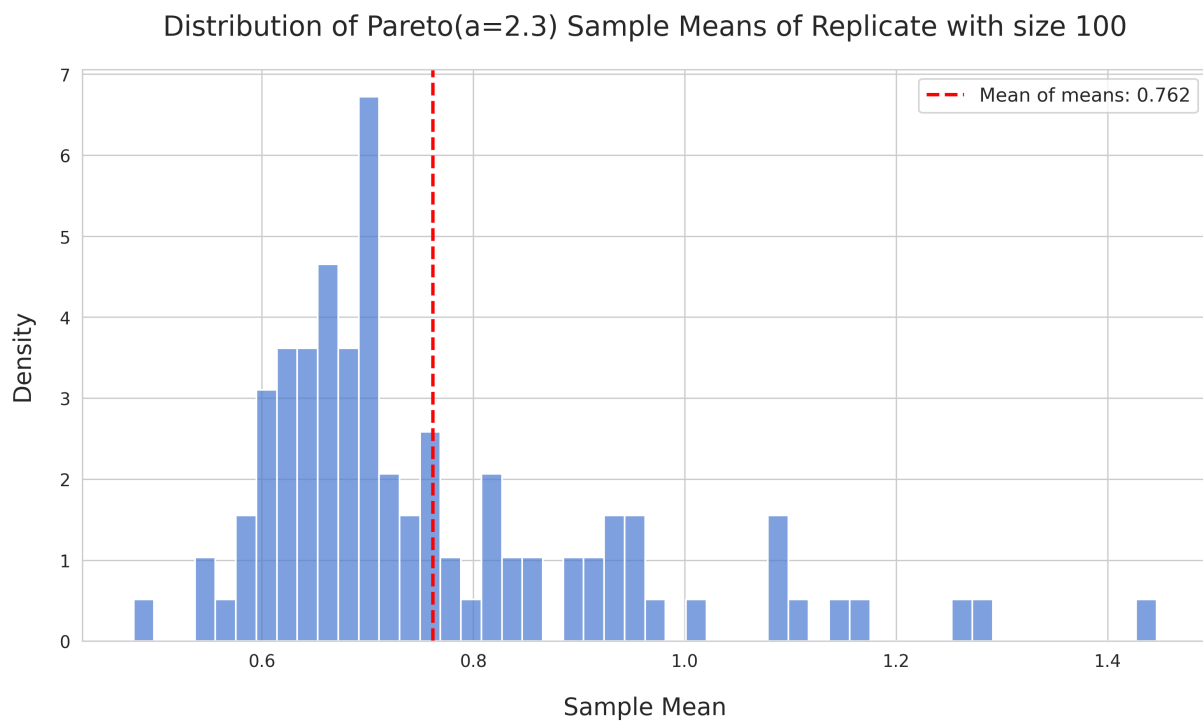


Figure 11: Distribution of sample means from Pareto($\alpha=2.3$) with replicate size 100.

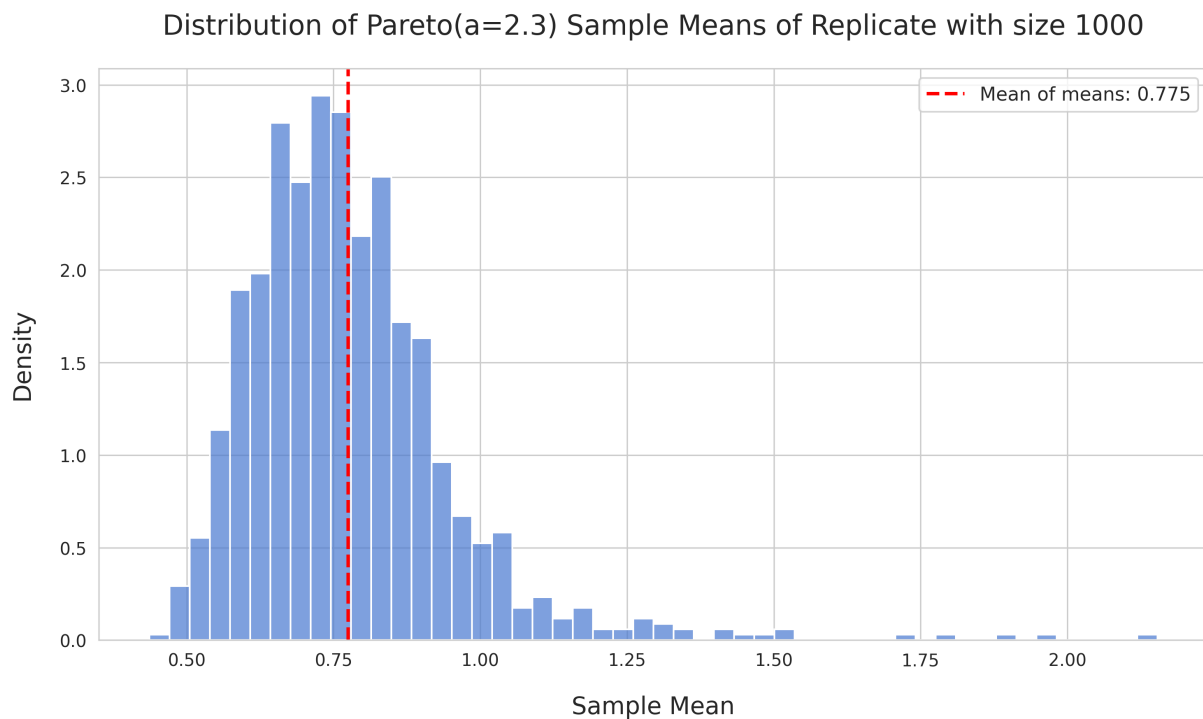


Figure 12: Distribution of sample means from Pareto($\alpha=2.3$) with replicate size 1000.

The Pareto sample mean plots show a similar convergence, though for the smaller two replicate sizes we have a number of datasets that tend to possess more of the predictable extreme values of the Power-Law distribution than others, causing a right skew in the distribution mass that mellows out only with increased number of sample means. The final plot shows a convergence toward a fairly smooth Gaussian, centered around 0.775, even if much slower due to its proclivity toward occasional extremes.

So regardless of the distribution type, we have at least a visual confirmation that CLT is applying appropriately to both, though the convergence of the sample means to a Gaussian as it relates to the number of samples may be influenced by the properties of the specific distributions in question.

3 Part C: Normal Approximation via CLT

- By the Central Limit Theorem, the distributions of sample means should approximate normal distributions. Determine the parameters (mean and standard deviation) of the approximating normal distributions for both power law and uniform cases.
- Overlay the probability density functions (PDFs) of these normal distributions on the corresponding plot from part B.

I computed the sample mean and the standard deviation of the sample means for each set of replicates, and then overlaid the PDF of the corresponding Gaussian defined with these (mean, std.) pairs on top of each of their respective plots from part B.

According to the Central Limit Theorem, for samples of size n drawn from a distribution with mean μ and variance σ^2 , the distribution of sample means should approximate a normal distribution with mean μ and standard error $SE = \frac{\sigma}{\sqrt{n}}$.

For the Uniform(0.25, 1.25) distribution, the theoretical parameters are:

$$\mu = \frac{0.25 + 1.25}{2} = 0.75$$

$$\sigma = \frac{1.25 - 0.25}{\sqrt{12}} \approx 0.289$$

$$SE = \frac{0.289}{\sqrt{100}} \approx 0.029$$

And the Pareto($a=2.3$) distribution:

$$\mu = \frac{1}{a-1} = \frac{1}{1.3} \approx 0.769$$

$$\sigma^2 = \frac{1}{(a-1)^2(a-2)} = \frac{1}{(1.3)^2(0.3)} \approx 1.97 \Rightarrow \sigma \approx 1.40$$

$$SE = \frac{1.40}{\sqrt{100}} \approx 0.140$$

The observed parameters from the sample means for each replicate size are shown below:

Observed Parameters from Sample Means (Mean μ , Standard Deviation σ)

Replicate Size	Uniform Distribution		Power Law Distribution	
	Mean (μ)	Std Dev (σ)	Mean (μ)	Std Dev (σ)
10	0.740	0.031	0.750	0.121
100	0.746	0.025	0.762	0.174
1000	0.750	0.029	0.775	0.176
Theoretical	0.750	0.029	0.769	0.140

For the Uniform distribution, the observed standard deviation stabilizes around 0.029, matching the theoretical standard error. For the Pareto distribution, the observed standard deviation increases from 0.121 to 0.176 as we get more replicates. The initial underestimation with 10 replicates likely occurs because we don't have enough samples to capture the full variability from the heavy tail, whereas the 1000 replicates provide a more realistic understanding of the variability caused by the occasional extremes. The observed means also converge: the Uniform mean reaches exactly 0.750 with 1000 replicates, and the Pareto mean reaches 0.775, fairly close to the theoretical 0.769.

This demonstrates that while the theoretical distribution of sample means is fixed (determined by the original distribution and sample size $n = 100$), we need a sufficiently high number of replicates to accurately estimate its parameters, especially for heavy-tailed distributions like the Pareto.

The function for computing and overlaying the normal PDFs is shown below:

```

def overlay_normal(sample_means: np.ndarray, dist_name: str,
                   replicate_size: int, output_filename: str):
    """
    Plots the histogram of sample means with overlaid normal PDF.

    :param sample_means: Array of sample means
    :type sample_means: np.ndarray
    :param dist_name: Name of the distribution
    :type dist_name: str
    :param replicate_size: Replicate size
    :type replicate_size: int
    :param output_filename: Output filename for figure
    :type output_filename: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")
    plt.figure(figsize=(10, 6))

    ax = sns.histplot(sample_means, bins=50, stat='density',
                      edgecolor='white', alpha=0.7, label='Sample Mean Distribution')

    mu = np.mean(sample_means)
    sigma = np.std(sample_means)
    x = np.linspace(min(sample_means), max(sample_means), 500)
    pdf = stats.norm.pdf(x, loc=mu, scale=sigma)

    plt.plot(x, pdf, 'r--', label=f'Normal Approx. PDF\(\mu={mu:.3f}, \sigma={sigma:.3f}\)')

    ax.set_xlabel('Sample Mean', fontsize=14, labelpad=15)
    ax.set_ylabel('Density', fontsize=14, labelpad=15)
    ax.set_title(f'Normal Overlay for {dist_name} Sample Means of Replicate Size {replicate_size}',
                 fontsize=16, pad=20)

    plt.legend(fontsize=11)
    plt.tight_layout()
    os.makedirs("../figures", exist_ok=True)
    plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)

```

Figure 13: Function for overlaying fitted normal PDF onto the sample mean distributions.

Normal Overlay for Uniform(0.25, 1.25) Sample Means of Replicate Size 10)

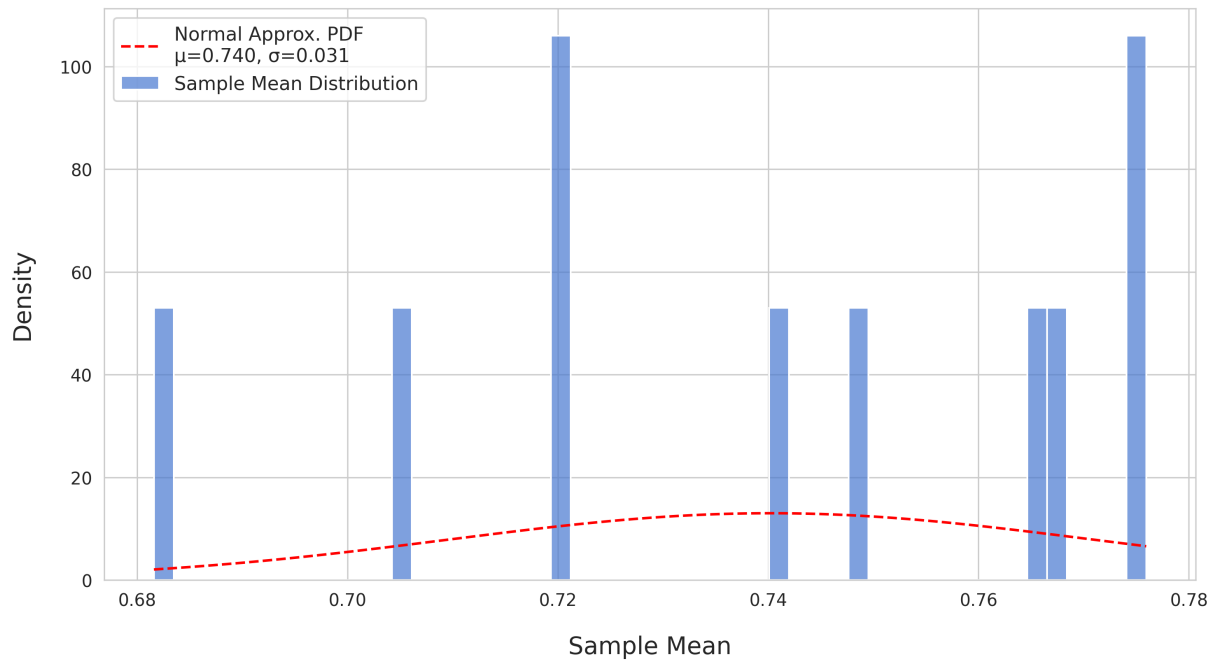


Figure 14: Uniform sample means, replicate size 10 with normal PDF overlay.

Normal Overlay for Uniform(0.25, 1.25) Sample Means of Replicate Size 100)

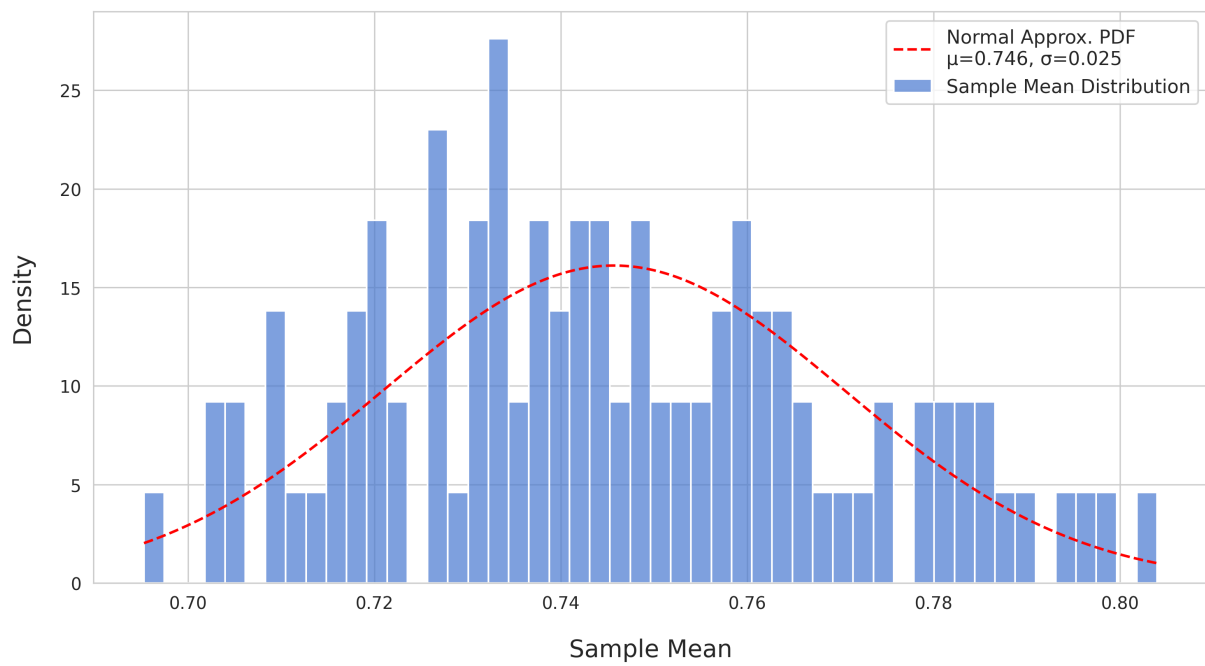


Figure 15: Uniform sample means, replicate size 100 with normal PDF overlay.

Normal Overlay for Uniform(0.25, 1.25) Sample Means of Replicate Size 1000)

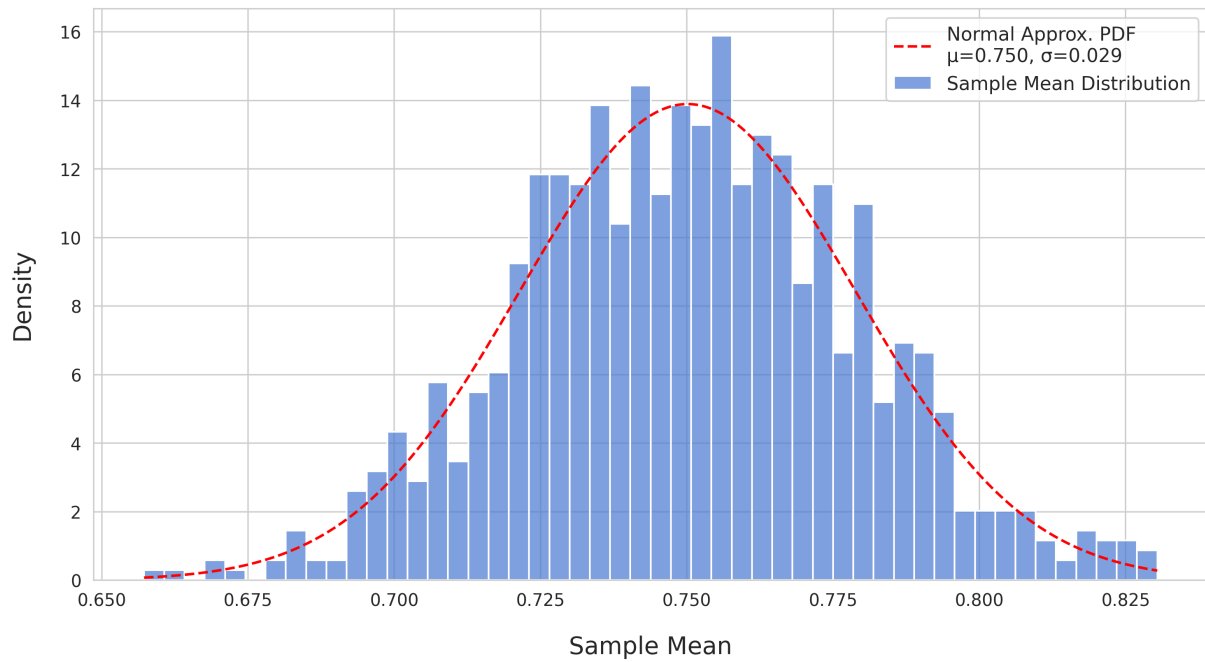


Figure 16: Uniform sample means, replicate size 1000 with normal PDF overlay.

Normal Overlay for Pareto($a=2.3$) Sample Means of Replicate Size 10)

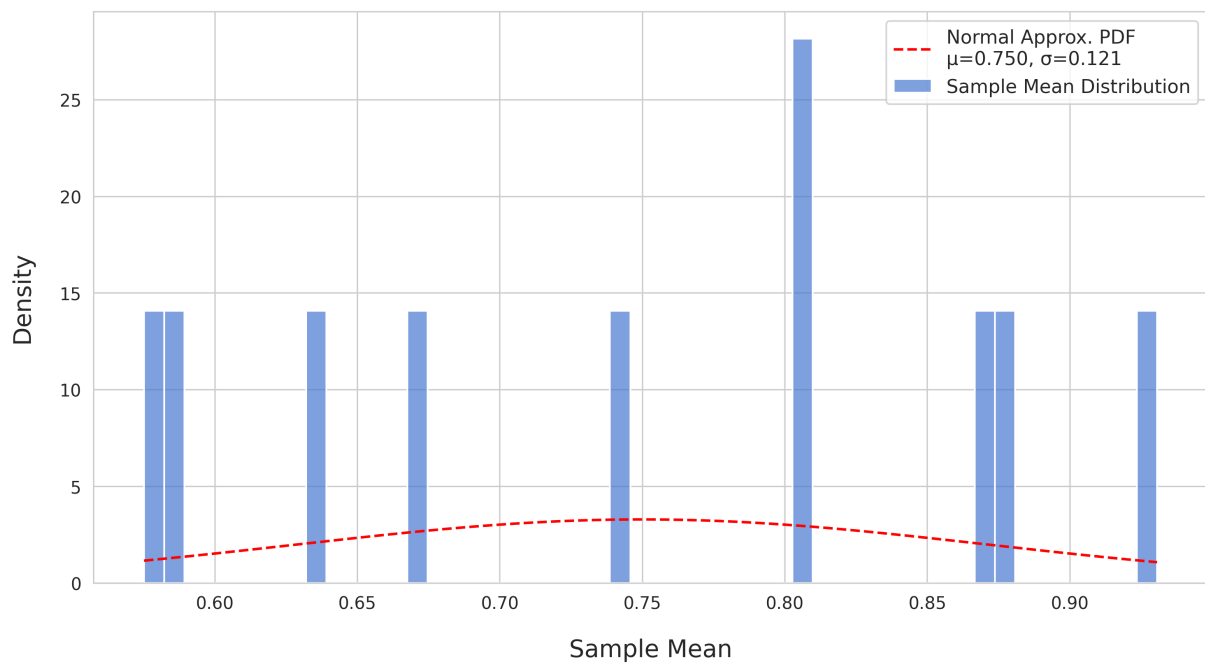


Figure 17: Power-law sample means, replicate size 10 with normal PDF overlay.

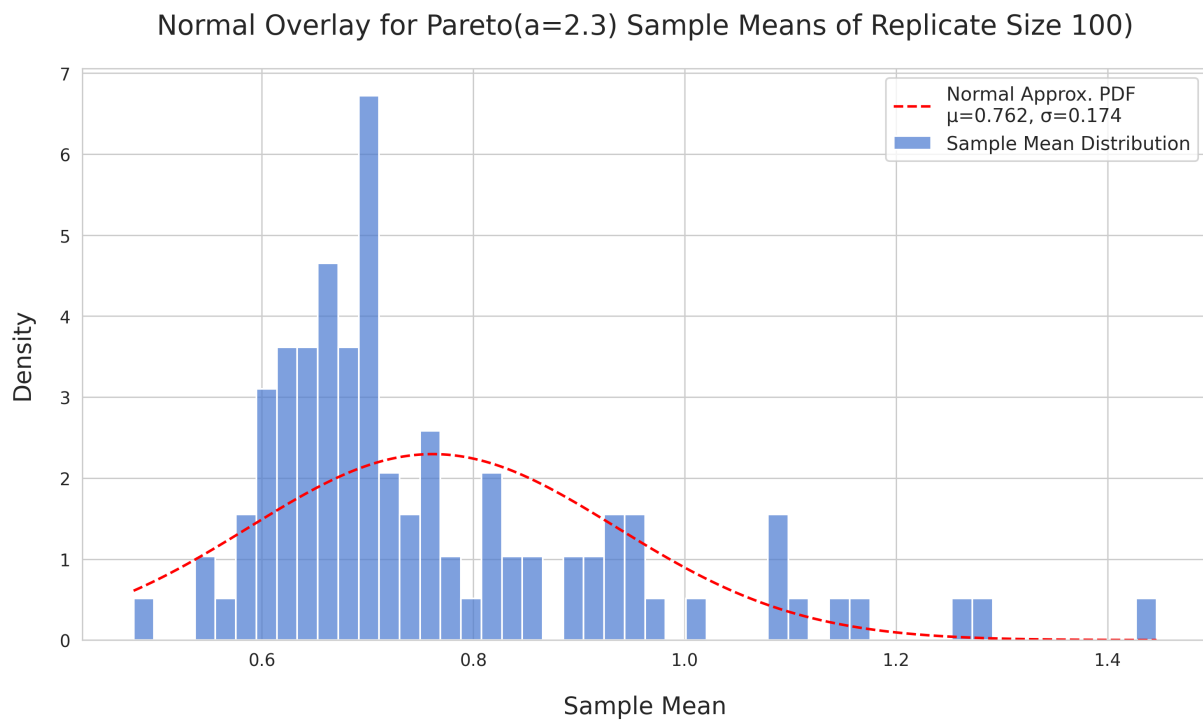


Figure 18: Power-law sample means, replicate size 100 with normal PDF overlay.

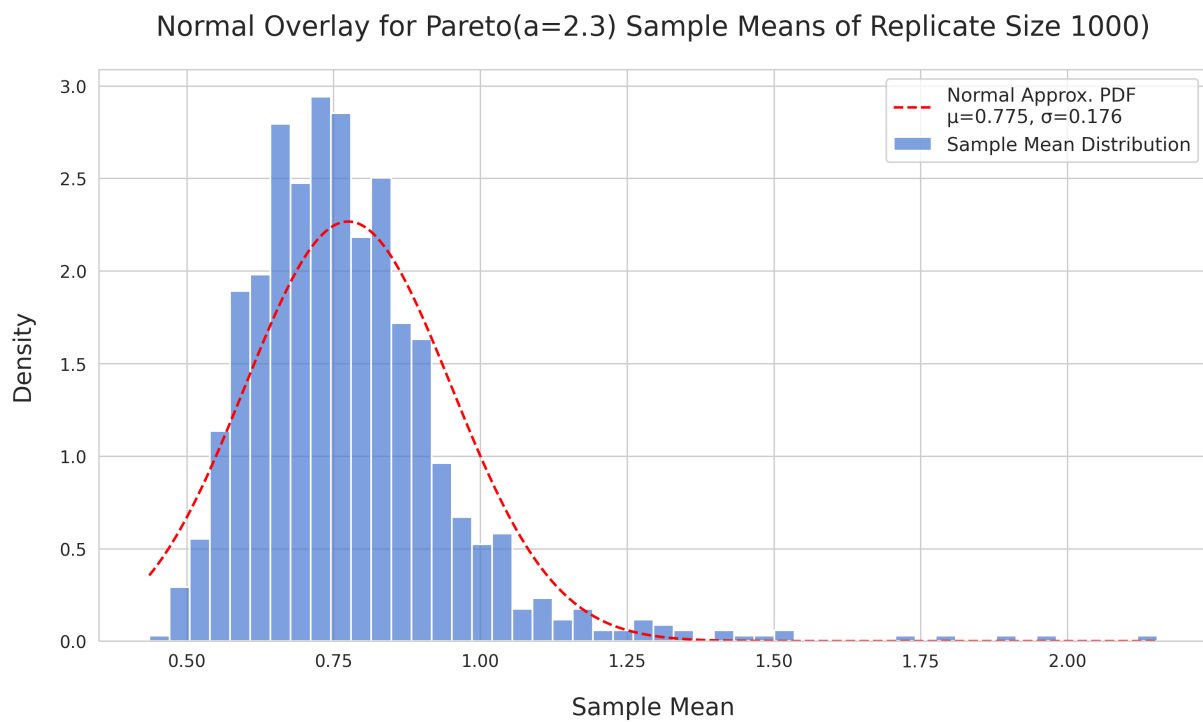


Figure 19: Power-law sample means, replicate size 1000 with normal PDF overlay.

4 Part D: Distribution Comparison with KS-Test

- Compare the empirical distributions of sample means (from power law vs. uniform) using the Kolmogorov–Smirnov (KS) test.
- Discuss whether the KS-test supports the theoretical expectation that both sample mean distributions should approximate normality under the CLT. And investigate how the power of the KS-test changes as the number of sample means increases (10, 100, 1000).

I performed one-sample KS tests comparing each distribution of sample means to its theoretical normal distribution. The implementation is shown below:

```
def ks_test_theoretical_normal(sample_means: np.ndarray, theoretical_mu: float,
                               theoretical_se: float) -> tuple:
    """
    Performs KS test comparing sample means to their theoretical normal distribution.

    :param sample_means: Array of sample means
    :type sample_means: np.ndarray
    :param theoretical_mu: Theoretical mean from CLT
    :type theoretical_mu: float
    :param theoretical_se: Theoretical standard error from CLT
    :type theoretical_se: float
    :returns: Tuple of KS statistic, p-value
    :rtype: tuple
    """
    ks_stat, p_value = stats.kstest(sample_means,
                                     lambda x: stats.norm.cdf(x,
                                                                loc=theoretical_mu,
                                                                scale=theoretical_se))

    return ks_stat, p_value
```

Figure 20: KS test function comparing sample means to theoretical normal distribution.

The one-sample KS test results for each distribution at different replicate sizes are tabulated below. Each test compares the empirical distribution of sample means to the theoretical normal distribution (Uniform: $N(0.75, 0.029^2)$, Pareto: $N(0.769, 0.140^2)$):

KS Test Results: Sample Means vs Theoretical Normal Distribution

Replicate Size	Uniform Distribution		Power Law Distribution	
	KS Statistic	p-value	KS Statistic	p-value
10	0.240	0.537	0.160	0.925
100	0.114	0.135	0.221	8.92e-5
1000	0.016	0.945	0.059	0.002

The Uniform distribution shows support for normality across each replicate size, with all the p-values being above 0.05, failing to reject the null hypothesis; the sample means ought to be following a Normal distribution. The KS statistics decreases from 0.240 to 0.016 with increasing replicates, so the agreement with the theoretical Normal distribution is increasing with greater numbers of sample means, as CLT would suggest.

The Pareto distribution reveals the effect of heavy tails on the KS test's power. At replicate size 10, the test fails to reject ($p = 0.925$) with a KS statistic of 0.160. This signals the limits of the KS test's power at smaller sample sizes because the KS stat at replicate size 10 is 0.16, which is larger than at replicate size 1000, despite the p-value at replicate size 1000 being significant while the other is not. For this reason, the KS test alone is not as powerful until sample size and statistical power at that sample size are taken into account, because in the case of size 10, the null could be false, but the test does not have enough power at this scale to detect it, whereas at size 1000 the test has so much power that it can detect any of the remaining deviations.

We see at size 100, the KS statistic actually increases to 0.221 and becomes highly significant ($p = 8.92 \times 10^{-5}$), rejecting the null hypothesis. This increase shows that with 100 replicates, the KS test has enough power at this scale to detect the sample's disagreement with the Gaussian from the heavy tail's influence on the sample mean distribution. This marks a turning point of the KS test becoming powerful with respect to our observations, and then the increase to size 1000 confirms the convergence, albeit the deviations from Normal behavior are still apparent and detectable by the test.

I also went ahead and plotted the convergence of the KS statistics for each distribution across the replicate sizes. Although the plot looks quite simple as a visual, it illustrates pretty well the behavior I just mentioned:

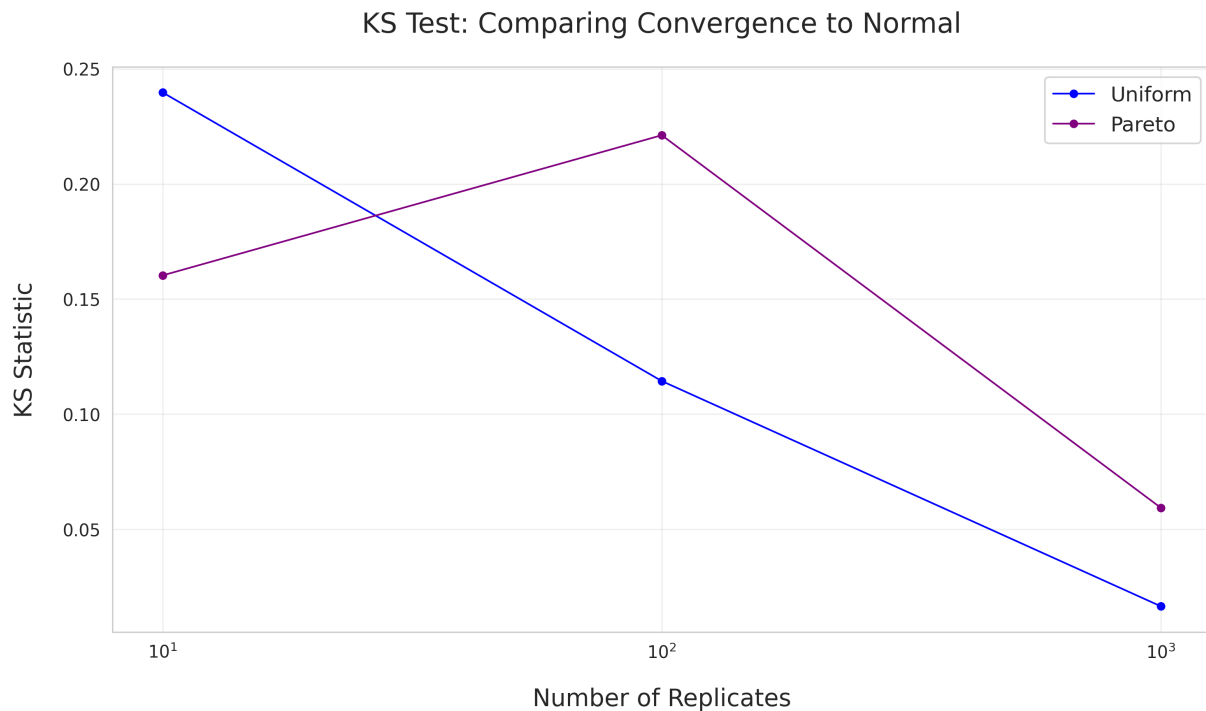


Figure 21: KS statistics vs replicate size, comparing Uniform and Pareto convergence.

Regarding the Uniform, it shows a consistently decreasing KS statistic, reaching almost zero by size 1000. It has excellent agreement with the Normal simply due to its construction: it doesn't have the issues in collecting sample means with similar behavior like the Pareto does. The Pareto shows an increase from 0.16 to 0.221 between size 10 to 100, and then decreases to 0.059 at size 1000. This is showing that initially, more data to reference the cumulative structure off of shows that deviations were present but undetectable. Then the CLT effect dominates over the tail behaviors of the Pareto causing the occasional extreme sample means.

The KS test results support the Central Limit Theorem's prediction that both distributions converge toward the theoretical Normal, though with the caveat that the rate of convergence will be influenced by the particular tail behaviors of the distributions in question.